

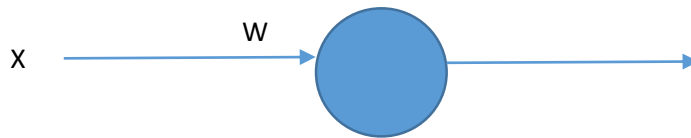
Gradient Descent

Introduction

Gradient descent is one of the most important supervised learning algorithms in the world of machine learning.

As we know, a model makes predictions by multiplying the input data by a set of weights. The values of these weights are crucial for producing accurate predictions.

The vector containing the input data is called the feature vector, while the vector of weights is known as the weight vector.



In this example, we have just a single neuron inside our neural network.

- X = the input value
- W = the weight

For our neural network to work properly, it needs to figure out the correct value to assign to W . This is exactly what gradient descent helps us calculate.

But before diving into gradient descent, it's important to understand what cost functions and loss functions are.

Loss function and cost function

A loss function is a function that takes as input (x, y, w) , where:

- x = the input
- y = the actual output associated with x
- w = the weight vector used (in this case, w is just a single value)

Using these parameters, since in our problem the output is a number, the loss function returns a value that represents the “distance” between the real output and the predicted output given by our model.

In short, it tells us how well (or poorly) our model is performing when trying to predict y from x using w .

There are many different types of loss functions. Here are a few examples:

- $\text{LossSquared} = (y_{\text{reale}} - y_{\text{predetta}})^2$
- $\text{AbsoluteLoss} = |y_{\text{reale}} - y_{\text{predetta}}|$
- $\text{LossFunction}(x, y, w) = \begin{cases} 0, & y_{\text{reale}} \neq y_{\text{predetta}} \\ 1, & y_{\text{reale}} = y_{\text{predetta}} \end{cases}$

Remember that:

$$y_{\text{predetta}} = x * w$$

Let's suppose we use the Squared Loss in our problem.

We can see that our model will work correctly if, for every possible input x , the loss function returns a value close to zero.

The function that gives us the overall error—and therefore actually evaluates how well our model is performing—is called the Cost Function.

In simple terms, the cost function is just the average of all the values returned by the loss function. This means we pass all the training data from our dataset into the loss function, and the cost function aggregates those results into a single number that represents the model's performance.

$$\text{CostFunction} = \frac{\sum_{x,y \in \text{trainingSet}} \text{SquaredLoss}(x, y, w)}{N}$$

Here, N is the number of data points in the training set, while in the summation, x and y vary. The lower the value returned by the cost function, the better our model is performing (meaning the values associated with the weight vector are closer to being correct).

In fact, the cost function itself also takes an input—the weights to be used. In other words, it receives the weight vector as a parameter.

Why have we spent time talking about the cost function and the loss function?

Because they are the very foundation of gradient descent. Without them, we wouldn't be able to calculate the correct values for the weights.

Now, let's move on to how gradient descent actually works.

Why is called gradient?

As we said earlier, our model works well if, when we pass the weight vector W to the cost function, the returned value is close to zero.

So, our goal is to find a W such that the cost function outputs an infinitesimal value.

In other words, we need to find a minimum of the cost function.

And how do we find the minimum of a function? By taking its derivative.

This is exactly the idea behind gradient descent: using derivatives and the cost function to locate a minimum (ideally the global minimum) of the cost function. Once we've found it, we've also found the correct values for W .

The process starts by initializing W either with all zeros or with random values. Then, we enter a loop that repeats N times (where N is a number we decide).

Now comes the tricky part. In our simple example, we only have a single weight to consider, so the output is just:

$$y = wx$$

Of course, things aren't always this simple. More complex problems require many more weights, which makes the output function much more complicated.

That's where the name gradient descent comes from.

When a function depends on multiple variables, we deal with partial derivatives—that is, we take the derivative with respect to one variable while keeping the others fixed.

The gradient is simply a vector that contains all these partial derivatives.

So, in order to adjust the values of all the weights, we need to compute the gradient of the cost function—meaning all of its partial derivatives.

Fortunately, in our example, we only have one weight, so we just need to calculate a single partial derivative: the one with respect to that weight.

$$(y_{\text{reale}} - y_{\text{predetta}})^2 = (y_{\text{reale}} - xw)^2$$

So we have to do the derivative of:

$$\begin{aligned} z &= (y_{\text{reale}} - xw)^2 \\ z' &= 2(y_{\text{reale}} - xw)(-x) \\ z' &= -2x(y_{\text{reale}} - xw) \end{aligned}$$

We've now found the partial derivative. Keep in mind that x and y will later be provided as inputs.

However, when we say x , we really mean all the data points in our training set.

So, the value of z' must be calculated using all the data in the training set.

The formula will therefore be:

$$z' = \frac{-2 \sum_{x, y_{\text{reale}} \in \text{trainingSet}} x(y_{\text{reale}} - xw)}{n}$$

To recap:

- We start by generating the weights randomly (in our case, we only have W).
- We enter a loop that repeats N times (with N chosen by us).
- We calculate the partial derivatives (based on the entire training set) with respect to each weight (since we only have one weight, we only need a single derivative).

How to adjust the weights

Now, the only thing left is to use this partial derivative to update the weight value, so that we gradually converge toward a minimum of the cost function.

The formula we need to use is the following:

$$w = w - \alpha(z')$$

Here, z' is the value of the partial derivative we calculated earlier, and α is the so-called learning rate (we'll come back to this later).

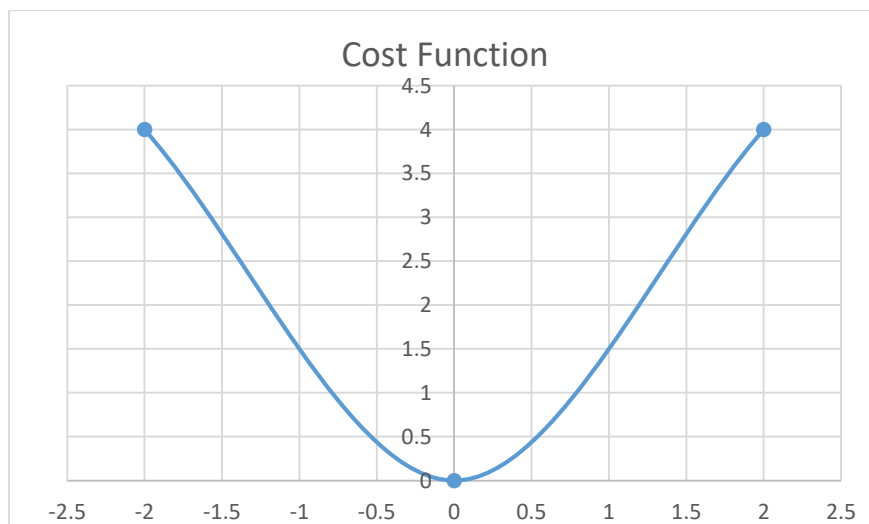
In this setup:

- If the derivative is positive, we subtract from the weight.
- If the derivative is negative, we add to the weight.

This adjustment is what allows us to converge toward a minimum:

- A negative derivative means the function is decreasing at that point,
- while a positive derivative means the function is increasing.

Example of Cost Function:



In this example, the minimum lies at $x=0$. Of course, this is just a simplified case to show how gradient descent works. In reality, as we saw earlier, the horizontal axis should represent the weight vector.

Before wrapping up, it's important to talk about the learning rate, denoted as α .

The learning rate is simply a number between 0.0 and 1.0 (inclusive) that determines how far we move left or right at each step. Looking back at the formula, you'll notice that this number is

multiplied by the derivative z' .

This scaling is crucial: without it, if z' is large, we would take steps that are too big, likely overshooting the minimum.

Choosing the learning rate is not arbitrary. Intuitively:

- The closer it is to zero, the slower we move.
- The closer it is to one, the faster we move (but with a higher risk of skipping over the minimum).

There's no universal rule for picking the perfect learning rate. A common strategy is to start with a relatively high value (sometimes even close to 1) and gradually decrease it as training progresses.

Problems with Gradient Descent

Gradient descent is one of the most popular supervised learning algorithms, but like everything else, it has its drawbacks. The main issue is that it can be too slow.

As we've seen, each partial derivative must be calculated over the entire training set, and this has to be repeated for every weight in the model. This makes the computation very expensive.

That's why other versions of gradient descent exist (though we won't dive into them here), such as Stochastic Gradient Descent and Mini-Batch Gradient Descent.

If you're curious, you can find implementations of these algorithms in the accompanying .py file.

Conclusion

In the end, after repeating the loop N times (with N likely being a very large number), we'll have found the correct values for W that minimize the errors in our predictions.